

Manual de Usuario — 3xtrat3rr3str3D

Guía práctica para **usar** el compilador 3xtrat3rr3str3D: desde la instalación hasta la escritura de programas en los tres lenguajes, la compilación y la ejecución del resultado.

1. ¿Qué es 3xtrat3rr3str3D?

Es un compilador que toma programas escritos en tres lenguajes de alto nivel inventados por la Resistencia:

Lenguaje	Extensión	Estilo
Lat (Pig Latin)	.pig	Imperativo; puede importar los otros dos
Y?	.y	Estilo Python (indentación); estructuras y funciones
Zetariano	.z	Orientado a objetos (una clase por archivo)

El compilador valida el programa y lo traduce a **código de tres direcciones (C3D)** y luego a un programa **C** que se compila con gcc para producir un **ejecutable**.

2. Requisitos e instalación

2.1 Requisitos

- **Java 17** (o superior) instalado.
- **Maven 3.9.x**.
- **gcc** en el PATH (se usa para compilar el C generado).
- Opcional: un IDE (IntelliJ IDEA, VS Code, etc.).

Verifica tu entorno:

```
java -version  
mvn -version  
gcc --version
```

2.2 Obtener el proyecto

Clona el repositorio y entra a la carpeta:

```
git clone <url-del-repositorio>
```

```
cd 3xtrat3rr3str3D
```

2.3 Compilar el proyecto

```
mvn -q compile
```

Si no hay errores, ya puedes ejecutar el compilador.

3. Inicio rápido

3.1 Desde la línea de comandos

```
mvn -q compile
```

```
CP="target/classes:$(mvn -q dependency:build-classpath -Dmdep.outputFile=/dev/stdout)"
```

```
java -cp "$CP" com.navi.Main testfiles/completo/main.pig
```

Verás la tabla de símbolos, las cuartetas C3D y, al final, las rutas del archivo .c y del ejecutable en output/.

3.2 Ejecutar el programa generado

```
./output/main
```

El ejecutable se llama igual que el archivo fuente, sin extensión, y se ubica en output / junto al fuente (en la CLI) o junto al archivo compilado (en la GUI).

3.3 Desde la interfaz gráfica

El arranque de la GUI está comentado en `Main.java`. Para habilitarla, descomenta el `main` que crea `CompilerWindow` (el bloque con `FlatMaterialDarkerIJTheme.setup()` y `new CompilerWindow()`), recompila y ejecuta `com.navi.Main`.

4. Tour de la interfaz

La ventana principal se divide así:

- **Barra de herramientas:** botones *Compilar*, *Guardar* y *Limpiar consola*.
- **Explorador de proyecto (izquierda):** árbol de carpetas y archivos.
- **Editor (centro):** pestañas; una por archivo abierto.
- **Resultados (abajo):** cuatro pestañas:
 - **Consola:** mensajes y errores.
 - **Tabla de símbolos:** variables, funciones, parámetros y su memoria.
 - **Tabla de tipos:** clases y estructuras con sus campos y miembros.
 - **C3D:** las cuartetas generadas.

5. Gestión del árbol de trabajo

5.1 Abrir un proyecto (carpeta)

1. Menú **Archivo** → **Abrir proyecto....**
2. Selecciona la carpeta de tu proyecto.
3. El árbol de la izquierda cargará los archivos `.pig`, `.y` y `.z`.
4. El título de la ventana mostrará el nombre de la carpeta.

5.2 Abrir un archivo

- **Doble clic** sobre el archivo en el explorador.
- El archivo se abre en una nueva pestaña del editor.

5.3 Crear archivos y carpetas

1. **Clic derecho** sobre una carpeta (o un archivo, para usar su carpeta).
2. Elige **Nuevo archivo** o **Nueva carpeta**.
3. Escribe el nombre. Los archivos nuevos deben terminar en `.pig`, `.y` o `.z`.

5.4 Eliminar

1. **Clic derecho** → **Eliminar**.
2. Confirma. Si es una carpeta, se elimina **todo su contenido** (irreversible).

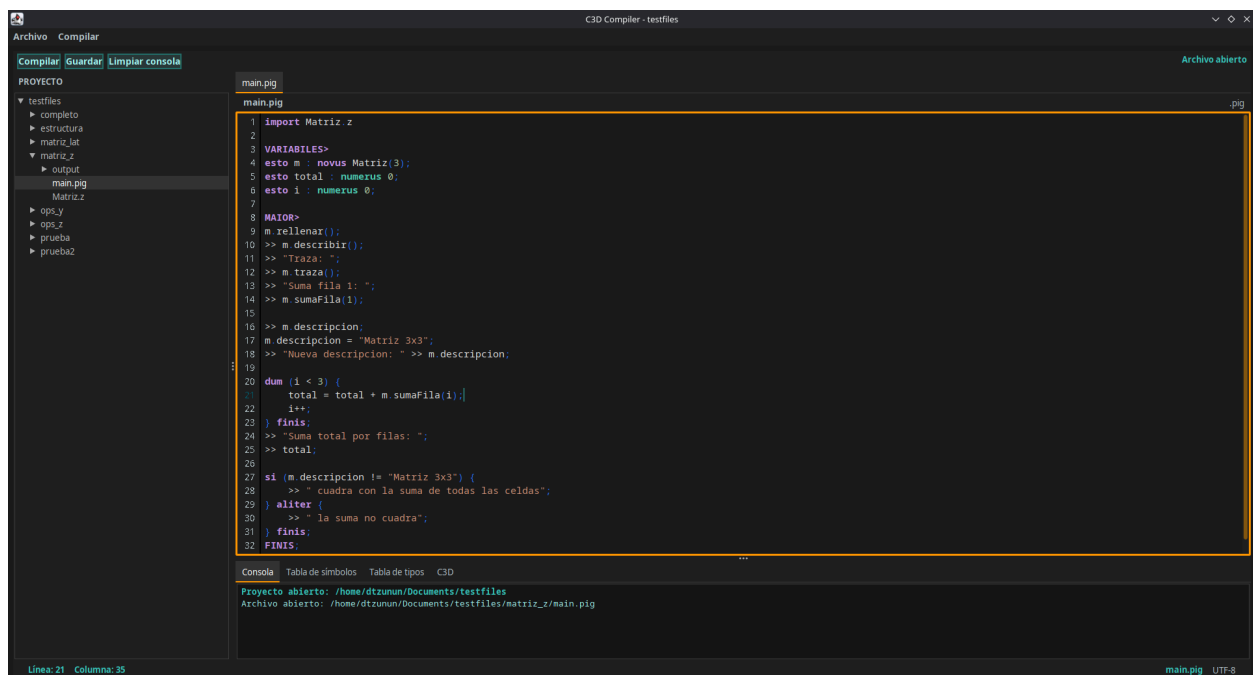
5.5 Guardar

- **Ctrl+S** o el botón **Guardar** guarda el archivo activo (sobrescribe).
- Al compilar, el archivo activo se guarda automáticamente antes de procesarse.

Nota: las operaciones de *renombrar*, *mover*, *copiar* y *descargar* no están incluidas en esta versión.

6. Editar código y resaltado

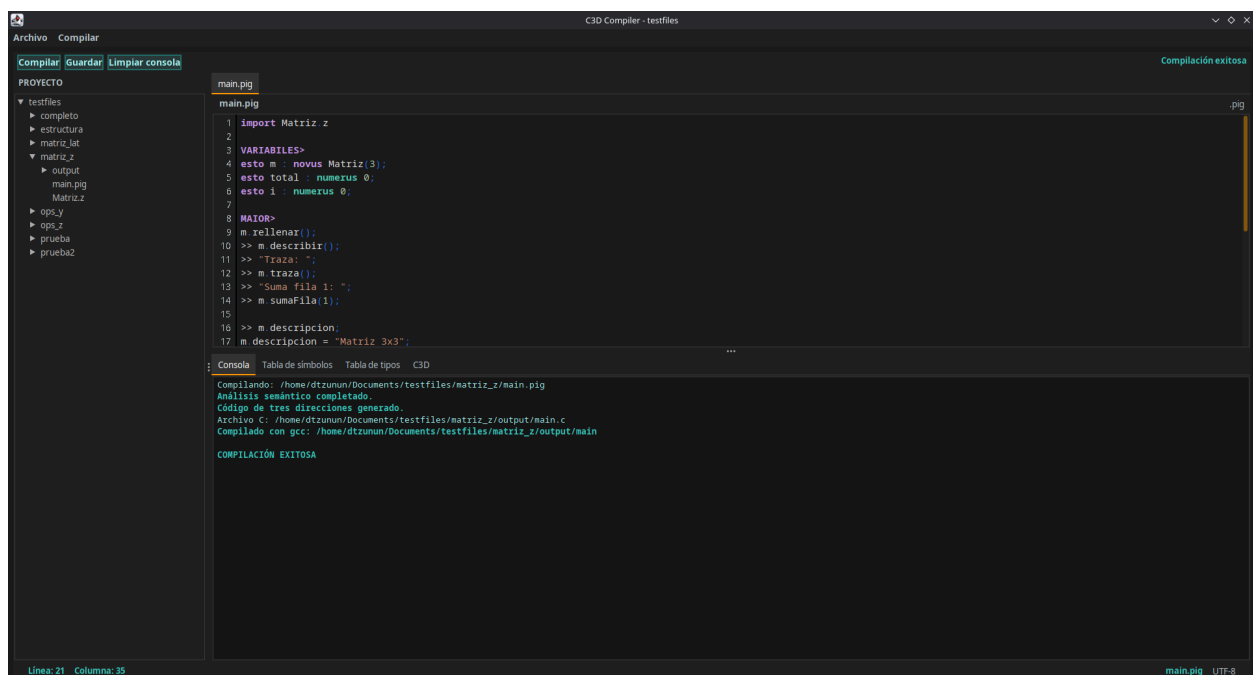
- El editor colorea el código **en tiempo real** (sin librerías externas):
 - Palabras clave, tipos, booleanos, números, cadenas, caracteres, comentarios, operadores y signos de puntuación tienen colores distintos.
- Los **números de línea** aparecen a la izquierda; la línea actual se resalta.
- **Tab** inserta 4 espacios.
- **Deshacer/Rehacer:** Ctrl+Z, Ctrl+Shift+Z o Ctrl+Y.
- **Cerrar pestañas:** clic con la rueda del ratón sobre la pestaña, o clic derecho para *Cerrar*, *Cerrar las demás* o *Cerrar todas*.



7. Compilar y ver resultados

1. Abre el archivo que quieres compilar (normalmente el `main.pig`).
2. Pulsa **Compilar** (barra de herramientas o menú **Compilar** → **Compilar archivo actual**).
3. Observa la **Consola**:
 - Si hay **errores**, verás una lista con el formato `[línea:columna] mensaje`. Las tablas y el panel C3D no se llenan.
 - Si es **exitoso**, verás:
 - Análisis semántico completado.
 - Código de tres direcciones generado.
 - Archivo C: `.../output/archivo.c`
 - Compilado con gcc: `.../output/archivo`
 - COMPILACIÓN EXITOSA
4. Revisa las pestañas **Tabla de símbolos**, **Tabla de tipos** y **C3D**.
5. El explorador se actualiza para mostrar la carpeta `output /` con el `.c` y el ejecutable.

La interfaz no ejecuta el programa. Para correrlo, abre una terminal y ejecuta el binario de `output /` (o compílalo tú mismo desde el `.c`).



ArchivoCompilar

CompilarGuardarLimpiar consola

PROYECTO

testfiles

completo

estructura

matriz_lat

matriz_z

output

main.pig

Matriz.z

ops_y

ops_z

prueba

prueba2

main.pig

main.pig

1 import Matriz.z

2

3 VARIABLES>

4 esto m : novus Matriz(3);

5 esto total : numerus 0;

6 esto i : numerus 0;

7

8 MAIOR>

9 m.rellenar();

10 >> m.describir();

11 >> "Traza: ";

12 >> m.traza();

13 >> "Suma fila 1: ";

14 >> m.sumaFila(1);

15

16 >> m.descripcion;

17 m.descripcion = "Matriz 3x3";

...

Consola

Tabla de símbolos

Tabla de tipos

CSD

Nombre	Kind	Tipo	Ámbito	Pos. memoria	Línea	Columna
m	VARIABLE	class Matriz	GLOBAL	0	4	0
total	VARIABLE	int	GLOBAL	1	5	0
i	VARIABLE	int	GLOBAL	2	6	0
n	PARAMETER	int	GLOBAL / FUNCTION 2	1	14	18
j	VARIABLE	int	GLOBAL / FUNCTION 3 / BLOCK 1 / LOOP 1	1	22	17
j	VARIABLE	int	GLOBAL / FUNCTION 3 / BLOCK 1 / LOOP 1 / BLOCK 1 / LOOP 1	2	23	21
suma	VARIABLE	int	GLOBAL / FUNCTION 4 / BLOCK 1	1	30	12
i	VARIABLE	int	GLOBAL / FUNCTION 4 / BLOCK 1 / LOOP 1	2	31	17
fila	PARAMETER	int	GLOBAL / FUNCTION 5	1	37	24
total	VARIABLE	int	GLOBAL / FUNCTION 5 / BLOCK 1	2	38	12
j	VARIABLE	int	GLOBAL / FUNCTION 5 / BLOCK 1 / LOOP 1	3	39	17
t	VARIABLE	int[]	GLOBAL / FUNCTION 6 / BLOCK 1	1	46	16
i	VARIABLE	int	GLOBAL / FUNCTION 6 / BLOCK 1 / LOOP 1	2	47	17
j	VARIABLE	int	GLOBAL / FUNCTION 6 / BLOCK 1 / LOOP 1 / BLOCK 1 / LOOP 1	3	48	21
texto	VARIABLE	string	GLOBAL / FUNCTION 7 / BLOCK 1	1	56	15
i	VARIABLE	int	GLOBAL / FUNCTION 7 / BLOCK 1 / LOOP 1	2	57	17
j	VARIABLE	int	GLOBAL / FUNCTION 7 / BLOCK 1 / LOOP 1 / BLOCK 1 / LOOP 1	3	58	21

Línea: 21Columna: 35

main.pigUTF-8

ArchivoCompilar

CompilarGuardarLimpiar consola

PROYECTO

testfiles

completo

estructura

matriz_lat

matriz_z

output

main.pig

Matriz.z

ops_y

ops_z

prueba

prueba2

main.pig

main.pig

1 import Matriz.z

2

3 VARIABLES>

4 esto m : novus Matriz(3);

5 esto total : numerus 0;

6 esto i : numerus 0;

7

8 MAIOR>

9 m.rellenar();

10 >> m.describir();

11 >> "Traza: ";

12 >> m.traza();

13 >> "Suma fila 1: ";

14 >> m.sumaFila(1);

15

16 >> m.descripcion;

17 m.descripcion = "Matriz 3x3";

...

Consola

Tabla de símbolos

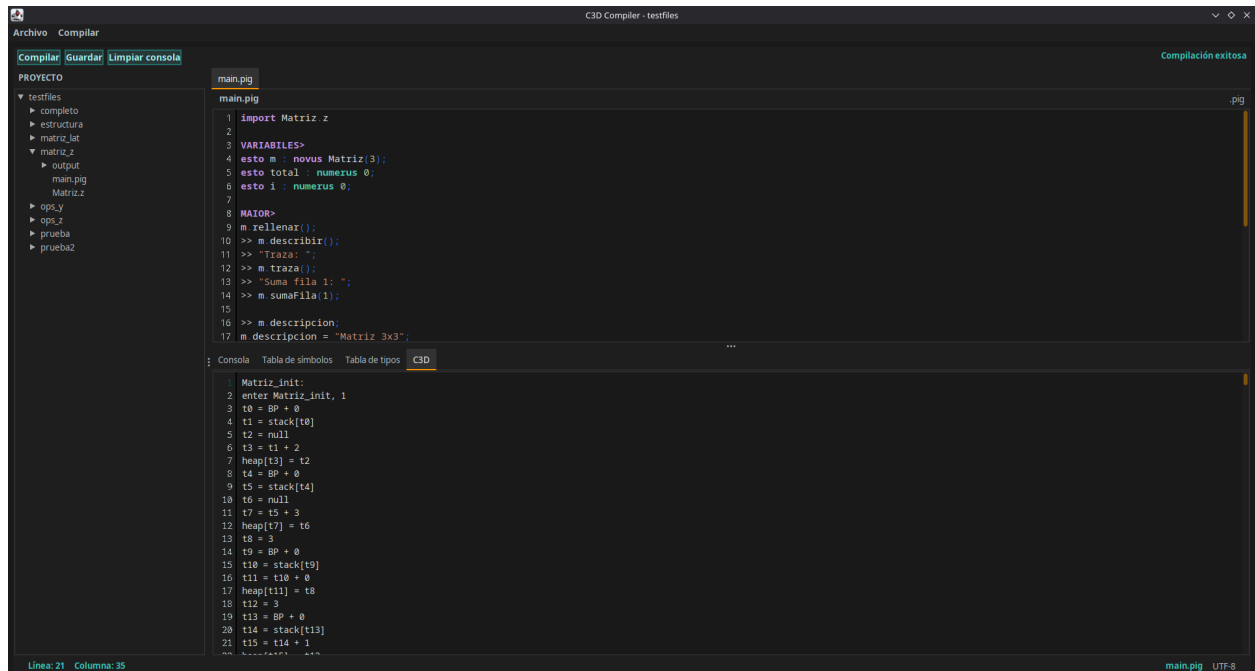
Tabla de tipos

CSD

Agregado	Categoría	Miembro	Tipo	Detalle
Matriz	class	flas	int	campo
Matriz	class	columnas	int	campo
Matriz	class	datos	int[]	campo
Matriz	class	descripcion	string	campo
Matriz	class	Matriz	class Matriz	constructor[] -> class Matriz
Matriz	class	rellenar	class Matriz	constructor[int] -> class Matriz
Matriz	class	rellenar	void	method[] -> void
Matriz	class	traza	int	method[] -> int
Matriz	class	sumaFila	int	method[int] -> int
Matriz	class	transponer	int[]	method[] -> int[]
Matriz	class	describir	string	method[] -> string

Línea: 21Columna: 35

main.pigUTF-8



8. Los tres lenguajes en 5 minutos

8.1 Lat (.pig) — el lenguaje de entrada

Un programa Lat tiene tres secciones: variables globales (opcional), funciones (opcional) y MAIOR> (obligatoria). Cierra con FINIS ;.

VARIABLES>

esto edad : numerus 20;

esto nombre : textum "Cadete";

MAIOR>

>> "Hola " >> nombre;

si (edad >= 18) {

 >> "Es mayor de edad";

} aliter {

 >> "Es menor de edad";

```
} finis;
```

```
FINIS;
```

Puntos clave:

- Declaración: esto nombre : tipo valor; y arreglos series x[n] : tipo {...};.
- Imprimir: >> expresión;. Leer: variable << (o solo <<).
- Funciones: actio nombre(params) { ... } finis; (sin retorno) y ratio tipo nombre(params) { ... } finis; (con retorno).
- Condicional: si (cond) { ... } aliter { ... } finis;.
- Ciclos: dum (cond) { ... } finis;; facere { ... } dum (cond);, per (esto i : numerus 0; i < 10; i++) { ... }.
- Objetos de Z: esto obj : novus Clase(args);.
- Imports: import carpeta.Archivo.z / import carpeta.archivo.y.

8.2 Y? (.y) — estilo Python

La **indentación** define los bloques. Hay sección de estructuras (opcional) y de funciones (obligatoria).

%estructuras

estructura Punto:

```
entero x
```

```
entero y
```

%funciones

definir sumar(entero a, entero b) -> entero:

```
retornar a + b
```

definir main():

```
entero r = sumar(3, 4)
```

```
imprimir("Resultado: ", r)
```

Puntos clave:

- Función: definir nombre(params) -> tipo: (el -> tipo es opcional).
- Estructura: estructura Nombre: y campos indentados.
- Arreglos [] entero arr y estructuras {} Punto p como parámetros se pasan **por referencia**.
- Condicional: si (...) entonces / sino (...) entonces / contrario.
- Selección: elegir(expr): con caso valor: y siempre:.
- Ciclos: para (...), mientras (...) hacer:, hacer: ... mientras (...).
- Control: romper, continuar, retornar.
- E/S: imprimir(...), leer().

8.3 Zetariano (.z) — orientado a objetos

Un archivo .z contiene **una clase**; el nombre del archivo debe coincidir con el de la clase.

```
public class Persona {

    String nombre;

    int edad;

    public Persona(String nombreParam, int edadParam) {

        nombre = nombreParam;

        edad = edadParam;

    }

    public void saludar() {

        println("Hola, me llamo " + nombre + " y tengo " + edad + " años.");

    }

    public int anioNacimiento(int anioActual) {

        return anioActual - edad;

    }

}
```

Puntos clave:

- Atributos con `public` opcional.
- Constructores y métodos pueden **sobrecargarse** (mismo nombre, distintos parámetros).
- `new Clase(args)` crea objetos en el **heap**; `new int[3][3]` crea arreglos.
- `null` para referencias vacías.
- Ternario: `cond ? a : b`.
- Asignación compuesta: `x += 3`, `x -= 2`, `x *= 2`.
- E/S: `print(...)` (sin salto), `println(...)` (con salto), `readln()`.
- Ciclos `for`, `while`, `do { ... } while (...)`; `switch/case/default`, `break`, `continue`.

8.4 Tabla comparativa rápida

Concepto	Lat	Y	Z
Entero	numerus	entero	int
Decimal	decimalis	flotante	double
Carácter	littera	caracter	char
Cadena	textum	cadena	String
Booleano	bool / verum / falsus	bool / verdadero / falso	boolean / true / false
Imprimir	<code>>> x;</code>	<code>imprimir(x)</code>	<code>print(x) / println(x)</code>
Leer	<code>x <<</code>	<code>leer()</code>	<code>readln()</code>
Si	<code>si (c) { } finis;</code>	<code>si (c) entonces</code>	<code>if (c) { }</code>
Mientras	<code>dum (c) { } finis;</code>	<code>mientras (c)</code> <code>hacer:</code>	<code>while (c) { }</code>
Función	<code>ratio tipo f() { }</code> <code>finis;</code>	<code>definir f() -></code> <code>tipo:</code>	<code>tipo f() { }</code>

9. Ejemplos guiados

Todos los ejemplos están en `testfiles/`. Puedes abrirlos y compilarlos.

9. Errores frecuentes

Mensaje / Situación	Causa probable	Solución
<code>[1:c] missing 'FINIS'</code>	Falta cerrar el programa	Agrega FINIS; al final del <code>.pig</code>
<code>[1:c] no viable alternative at input ...</code>	Sintaxis incorrecta	Revisa la línea; recuerda que Y depende de la indentación
Tipo no definido: X	Usaste una estructura/clase no importada	Agrega el import correspondiente (en Lat)
Variable duplicada: x	Declaraste dos veces el mismo nombre en el mismo ámbito	Renombra o elimina una
No se puede asignar <code>double</code> a <code>int</code>	Intento de degradar un decimal	Usa una variable <code>decimalis/double/flotante</code>
La condición debe ser booleana	Usaste un número/string en <code>si/if</code>	Usa una comparación (<code>x > 0</code>) o un booleano
Extensión de import no válida	El import no termina en <code>.y</code> o <code>.z</code>	Corrige la ruta
No se pudo cargar el módulo	Ruta del import incorrecta	Verifica la ruta relativa a la carpeta del <code>.pig</code>
<code>gcc terminó con código 1</code>	Error del C generado o falta <code>gcc</code>	Asegúrate de tener <code>gcc</code> ; revisa la consola
El ejecutable no se ejecuta	No diste permisos o la ruta es distinta	Ejecuta <code>./output/<nombre></code> desde la raíz del proyecto

11. Preguntas frecuentes

¿Qué archivo debo compilar si tengo varios? El `.pig` principal. Ese archivo importa los `.y` y `.z` que necesite.

¿Puedo compilar un `.y` o `.z` solo? Sí, Main acepta `.y` y `.z` directamente (útil para probar un módulo).

¿Dónde queda el código C y el ejecutable? En la carpeta `output/`. En la CLI, junto al archivo fuente; en la GUI, junto al archivo compilado. La carpeta se limpia en cada compilación.

¿Puedo ver las cuartetas generadas? Sí, en la pestaña **C3D** de la interfaz o en la salida de la línea de comandos.

¿Por qué mi código Y da error de sintaxis si "se ve bien"? Y es sensible a la **indentación**: usa espacios/tabulaciones consistentes y `:` al final de las líneas que abren bloque.

¿Se permite herencia/polimorfismo en Z? No en esta entrega; están reservados para el Proyecto 2.

¿Necesito instalar algo para ejecutar el resultado? Solo gcc para compilar; el ejecutable resultante ya es nativo.

¿La interfaz ejecuta el programa? No. Compila y reporta las rutas; ejecuta el binario desde una terminal.